

Métaheuristiques

Master 1 Informatique

Projet – Courbes de Bézier

Indications générales

Date de rendu : 5 mai 2026 avant 8 h 00

Travail en groupes de deux

Les fichiers seront déposés sur ARCHE dans une archive `.zip`

Création d'un itinéraire avec courbes de Bézier

Le problème consiste à générer un itinéraire optimal dans un environnement donné, la trajectoire étant représentée à l'aide de courbes de Bézier. L'objectif est de concevoir une trajectoire continue qui relie un point de départ à un point d'arrivée, tout en respectant des contraintes spatiales (zone valide) et en évitant des obstacles.

Quatre instances sont mises à disposition pour le développement du projet et l'évaluation de la solution proposée. Quatre autres instances, tenues secrètes, seront utilisées uniquement lors de l'évaluation finale.

Le projet

L'objectif du projet est de développer un algorithme métaheuristique capable de générer une trajectoire (représentée par une courbe de Bézier) qui optimise un critère combinant la longueur de l'itinéraire, la fluidité de la courbe et le respect des contraintes (itinéraire dans la zone définie et évitement des obstacles). Chaque projet disposera de 60 secondes de calcul pour trouver une solution. Étant donné le caractère stochastique des méthodes proposées, une moyenne sur 10 exécutions sera utilisée pour évaluer la qualité des solutions obtenues.

Le code existant

Le projet devra être développé en langage Java en se basant sur les classes fournies. Tout le code développé se trouvera dans un **unique** *package* nommé `bezier.projects.competitor.nomduprojet` qui devra être créé. Ce *package* devra contenir **une seule classe étendant** `CompetitorProject.java`. L'**unique** constructeur de la classe devra nécessairement comporter **un seul** paramètre de type `Problem` et appeler obligatoirement et uniquement les méthodes `addAuthor` et `setMethodName` afin d'identifier le groupe d'étudiants pour l'évaluation du projet.

La classe `Project.java` contient deux méthodes abstraites qu'il faut définir. La méthode `initialization` est exécutée une fois au début de l'optimisation. L'initialisation devra être effectuée dans cette méthode et non dans le constructeur. La méthode `loop` représente la boucle principale de l'algorithme et sera répétée pendant le reste des 60 secondes. La classe `Project.java` contient également un attribut important, `Problem problem`, qui contient la fonction d'évaluation qui mesure la qualité

de la trajectoire ainsi que quelques informations sur le problème (bornes, points de départ et d'arrivée, nombre d'obstacles).

Une solution potentielle pourra prendre différentes formes (`double [][]`, `double []`, `Coordinates []` ou `ArrayList<Coordinates>`) afin de représenter l'ensemble des points de contrôle internes (excluant les points d'ancrage de départ et d'arrivée qui sont fixes) et pourra être évaluée via les méthodes `evaluate(...)` de la classe `Problem`. Ces méthodes mettent automatiquement à jour la meilleure solution découverte au cours de l'exécution. Toute évaluation appelée après le délai imparti sera ignorée. Il **n'est pas** autorisé d'utiliser des bibliothèques Java implémentant des algorithmes métaheuristiques. D'autres classes peuvent, cependant, être créées dans le *package*.

Les étudiants pourront s'inspirer du package `bezier.projects.demo` et des classes qui s'y trouvent.

Le travail demandé

Il est demandé de réaliser, en langage de programmation Java, un unique algorithme d'optimisation métaheuristique qui génère une trajectoire satisfaisante pour le problème de création d'itinéraire avec courbes de Bézier. Il n'est pas autorisé de faire du calcul distribué, que ce soit en local (*multi-thread*) ou à distance (serveurs de calculs), de charger des informations autres que les données fournies, ni d'instancier les algorithmes adverses.

Il conviendra d'utiliser l'encodage UTF-8 (notamment pour la gestion des caractères accentués). **Seul** le dossier contenant le *package* `bezier.projects.competitor.nomduprojet` devra être envoyé. Le code devra être rendu dans une archive `.zip`. **Aucune** modification du code existant n'est tolérée.

Un rapport de dix pages maximum est également demandé. Ce rapport devra être convenablement structuré (page de garde, introduction, conclusion, etc.). Il devra décrire la recherche bibliographique effectuée et détailler le principe de la méthode d'optimisation métaheuristique mise en œuvre.

La notation

La notation sera décomposée en quatre parties :

- l'ingéniosité et la pertinence de la solution mise en œuvre (9 points) ;
- la qualité de la programmation en langage Java (3 points) ;
- la qualité de la rédaction du rapport (3 points) ;
- l'efficacité face aux solutions proposées par les autres groupes d'étudiants sur les quatre instances du problème (5 points).

Les méthodes des différents groupes d'étudiants seront comparées entre elles. Le classement obtenu déterminera la note, allant de 1 (dernier du classement) à 5 (premier du classement). Une note de 0 sera attribuée aux groupes disqualifiés (méthodes non conformes au cahier des charges).

Remarques

Les questions relatives au projet doivent être adressées par e-mail à l'adresse suivante : `alexandre.blansche@univ-lorraine.fr`.